Memory Hardening & LLMs: A Newcomer's Study Guide

Oct 6, 2026 · @Shaurya

0. How to use this guide

The project asks one question: hardware defenses like MTE and PAC make exploitation "probabilistically hard" for humans, but does that still hold when an LLM agent is the attacker? Everything below builds the vocabulary and the mental models you need to discuss that question with Marius on Wednesday.

The one-paragraph version. C and C++ programs have memory bugs. Attackers turn those bugs into control over a device. Over decades, defenders added software and hardware barriers. The newest barriers (Arm's Memory Tagging Extension and Pointer Authentication) work by attaching small secret values to memory and pointers, so a wrong guess crashes the program. Because the secrets are small (4 bits for MTE, roughly 7 to 16 bits for PAC), a lucky guess is possible but unlikely. Marius wants to know whether an LLM changes that math in practice, by lowering the cost of attempts, spotting leaks, or finding places where the defense is not actually applied.

How I labeled things. I read both of Marius's articles in full today (October 6, 2026). Claims taken from them are marked *Apple says* or *Android says*. Claims from my own background knowledge are marked *Background*. My own reasoning is marked *Inference*. Things to ask Marius are marked *Ask*.

What I did not independently verify. The Codex brief you pasted cites several other sources (Marius's website, his papers, Linux and Clang documentation, the PACMAN, TikTag and StickyTags papers). I use its claims where noted, but treat paper details as "to confirm when you read them."

Reading paths by time available

Time you have	Read these sections	Skip for now
30 minutes	0, 2, 6, 10, 12	Everything else
2 hours	0 through 4, 6, 7, 10, 12	5, 9, 11 (skim)

Time you have	Read these sections	Skip for now
Full evening	All, in order	Nothing

How the sections connect

Sections 1 to 4 are the foundation: security vocabulary, memory bugs, how bugs become exploits, and the defenses that came before. Sections 5 to 8 are the technology Marius named: allocators and Scudo, MTE, PA, and Apple's combined system. Section 9 covers known attacks on those defenses. Section 10 is the research question itself. Section 11 is about Marius and his papers, and section 12 is meeting prep. Section 13 is a glossary you can search any time a word trips you up, and section 14 holds the reading order and sources.

1. Security vocabulary for a newcomer

Security papers are mostly careful statements of who can do what to whom. If you master six words (asset, attacker, vulnerability, exploit, mitigation, threat model) you can read most of the abstracts you will meet.

The core words

Term	Plain meaning	Example in this project
Asset	What the defender wants to protect	A phone user's messages and photos
Attacker	Whoever is trying to break in, plus what they can already do	An app the user installed, or a malicious web page
Vulnerability (bug)	A mistake in the program that could be abused	Writing 17 bytes into a 16-byte buffer
Exploit	Code or input that turns a vulnerability into attacker benefit	A crafted image file that makes the phone run attacker code
Mitigation	A defense that makes exploiting a bug harder without removing the bug	MTE, PAC, ASLR

Term	Plain meaning	Example in this project
Threat model	The written assumptions about attacker abilities, goals and limits	"Attacker has a memory-write bug and can retry freely, but cannot read tags"
Attack surface	Every place attacker-controlled input enters	Web content, messages, file parsers, system calls
Privilege boundary	A line the attacker wants to cross	App to kernel, browser tab to the rest of the OS

Ideas that matter more than the definitions

A mitigation is not a fix. The bug is still in the code. The mitigation changes what the bug lets an attacker do. This is why Apple's article keeps talking about making chains "more expensive and difficult to develop and maintain" rather than impossible. *Apple says* its goal is to disrupt mercenary spyware, which chains several bugs together at a cost of millions of dollars.

An exploit chain is a sequence of steps. A realistic attack might start with a bug in a web page renderer, escape the renderer's sandbox, then attack the kernel. Each step needs its own bug and its own technique. *Apple says* a defense that breaks one early step can invalidate the whole chain, and the attacker must start over with new bugs.

Deterministic versus probabilistic defenses. A deterministic defense blocks an attack outright, for example a guard page that always faults. A probabilistic defense blocks it most of the time, and the attacker wins only by luck. MTE with 4-bit tags and PAC with short signatures are probabilistic. That is exactly why Marius's question is interesting.

A crash is information. When a wrong guess crashes the program, the attacker learns something (the guess was wrong) and the defender also learns something (an attack happened, logs are written). Whether the attacker can try again depends on whether the target restarts, and whether the secret changes after a restart. Much of the research literature is really about this one question: does a failed attempt cost the attacker anything?

An oracle is anything that answers a question about a secret. If an attacker can ask "is this tag correct?" and get an answer without crashing, that is an oracle, and it turns a 1-in-16 gamble into a cheap search. Speculative execution attacks (section 9) are oracles that do not crash the process.

Reliability matters as much as possibility. An exploit that works 1 time in 16 and crashes the other 15 is very different from one that works every time. Papers report success rates and time to success, not just "it works."

Names you will see in papers

- **CVE:** an identifier for a specific publicly known vulnerability.
- **CWE:** a catalog of bug types. CWE-787 is out-of-bounds write. CWE-416 is use-after-free.
- **PoC (proof of concept):** minimal code showing a bug or attack works.
- **Sandbox:** an environment that restricts what code can do, so compromising it does not immediately compromise the system.

2. Memory safety bugs from first principles

Every memory safety bug is a violation of one of two questions: is this access inside the object it was meant for, and is that object still alive? The first is a spatial question and the second is a temporal one. MTE exists to catch both.

Background: what memory looks like to a program

Memory is a long array of bytes, each with a numeric address. A **pointer** is a variable that holds an address. On a 64-bit phone, a pointer is 64 bits wide.

Programs get memory in two main places. The **stack** holds local variables and return addresses, and is managed automatically as functions are called and return. The **heap** holds memory a program requests explicitly with `malloc` (C) or `new` (C++) and releases with `free` or `delete`. Heap bugs are the focus of this project, because the heap allocator (section 5) is where MTE tagging happens.

C and C++ do not check that a pointer stays inside its object, and do not track whether the object is still alive. That freedom is why they are fast, and why they have these bugs.

Spatial errors: going out of bounds

```
char *buf = malloc(16);   // valid indices: 0..15
buf[16] = 'A';            // one byte past the end: out-of-bounds write
```

The write lands on whatever sits next to the buffer. That might be another heap object, allocator bookkeeping, or unused padding. Writes are usually more dangerous than reads because they change program state, though out-of-bounds reads can leak secrets such as pointers or keys.

A common real-world cause is length arithmetic: a program computes a size, the computation overflows or is attacker-influenced, and a copy writes too many bytes.

Temporal errors: use after free

```
struct User *u = malloc(sizeof *u);
free(u);                  // the allocator may now reuse this memory
struct Msg *m = malloc(sizeof *m);  // may land at the same address as u
u->is_admin = 1;          // stale pointer now writes into m
```

The pointer `u` still holds the old address. The program has a **dangling pointer**. If the allocator gave the same memory to a new object, the stale write corrupts the new object. Setting `u = NULL` only fixes that one pointer. If other pointers (aliases) still refer to the same memory, they remain dangling.

Other bug types you will hear about

- **Double free:** freeing the same allocation twice, which can corrupt allocator state.
- **Type confusion:** treating an object as the wrong type, so field offsets mean something different.
- **Integer overflow:** arithmetic that wraps around and leads to a too-small allocation or a too-large copy.
- **Uninitialized memory:** reading bytes that were never set, which can leak old data.
- **Intra-object overflow:** overflowing from one field into another field *inside the same allocation*. This one matters a lot later, because tag checks cannot see it (section 6).

Why not just switch languages?

Memory-safe languages like Rust and Swift check bounds and lifetimes. *Apple says* it uses Swift for new code and targeted rewrites, while also deploying mitigations at scale. Billions of lines of existing C and C++ are not going away soon, which is why hardware-assisted hardening matters. (Marius's Crustify project, according to the Codex brief, uses an LLM harness for Rust migration.)

The five-stage vocabulary

The Codex brief proposes a vocabulary I recommend adopting, because it prevents sloppy claims:

1. **Bug:** the program violates its intended memory rules.
2. **Trigger:** an input or sequence that reaches the bug.
3. **Primitive:** a useful capability the bug provides, such as "write 8 controlled bytes at an adjacent object."
4. **Security effect:** the capability crosses a stated security boundary.
5. **Reliable exploit:** the effect is reproducible under the declared conditions.

A crash alone is only stage 1 or 2. A tag-fault means the defense worked. A "successful bypass" has to get to stage 4 at least.

3. From bug to exploit

An exploit is the engineering work of turning a small mistake (one bad write) into a reliable capability (running attacker code or changing a security decision). Defenses like MTE and PAC are designed to break specific steps of that process, so you need a rough picture of the steps.

Primitives: what a bug gives the attacker

Security researchers describe a bug by the capability it hands over, called a **primitive**:

- **Relative write:** write attacker-chosen bytes at some offset from a known object (a typical overflow).
- **Arbitrary read:** read any address the attacker names. This defeats secrecy.
- **Arbitrary write:** write any bytes to any address the attacker names. This is usually game over.
- **Information leak:** learn an address or secret value without any corruption.

Exploit development is largely about *upgrading* a weak primitive into a stronger one. A one-byte relative write might be used to corrupt a length field, which then allows a bigger read, which leaks a pointer, which enables an arbitrary write.

Heap grooming (heap feng shui)

Because the heap allocator decides where objects go, attackers manipulate the sequence of allocations and frees so the victim lands where they want. A typical use-after-free recipe:

1. Trigger the allocation of a victim object of some size.
2. Trigger its free while a dangling pointer remains.
3. Allocate many attacker-controlled objects of the same size, hoping one reclaims that slot.
4. Use the dangling pointer. It now reads or writes the attacker's object, interpreted as the victim type.

Apple says the attacker's goal in both use-after-free and out-of-bounds bugs is to create "overlapping interpretations of memory" by controlling where certain allocations land. This is precisely what typed allocators and tagging try to prevent.

Two flavors of end goal

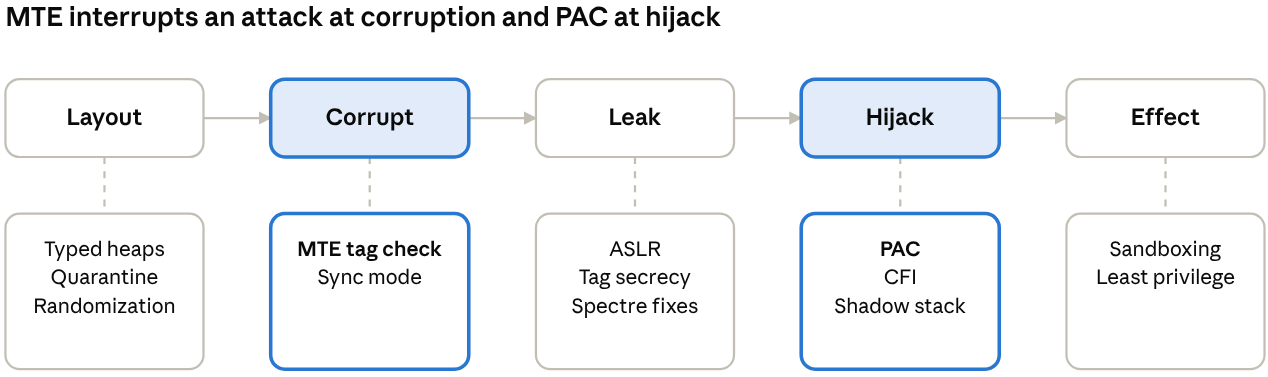
Control-flow hijack. Overwrite something that decides where the program jumps next: a return address, a function pointer, or a C++ virtual table entry. Modern attackers rarely inject new code. They reuse existing code fragments in a technique called return-oriented programming (ROP). *Background:* Pointer Authentication was designed to make forged return addresses and function pointers fail.

Data-only attack. Change a variable that the program trusts, such as a permission flag, a file path, or an object's length, without redirecting execution at all. This is important for the project: a defense that protects code pointers does nothing against a corrupted `is_admin` field unless something else protects that field. The Codex brief calls this a "data-only effect."

The role of information leaks

Many defenses depend on a secret the attacker does not know: where code lives (ASLR), the PAC signature, the MTE tag. Attackers therefore often need a leak first. In MTE research, the key question is whether tags can be learned without causing a fault. Apple's article spends a full section on this, which it calls Tag Confidentiality Enforcement.

Where each defense interrupts the pipeline



Top row: attacker steps. Bottom row: defenses aimed at each step. Highlighted: the two this project studies.

attack pipeline and the defenses aimed at each step

Read left to right: a defense that breaks an early step forces the attacker to restart with a different bug. The table below adds what the attacker does at each step.

Stage	What the attacker does	Defense aimed at it
Layout	Groom the heap so objects are adjacent or reused	Typed allocators, randomization, quarantine
Corrupt	Perform the out-of-bounds or stale access	**MTE** (tag mismatch faults)
Leak	Learn addresses and secrets	ASLR, tag confidentiality, side-channel mitigations
Hijack	Redirect execution	**PAC**, control-flow integrity
Effect	Gain code execution or change a security decision	Sandboxing, privilege separation

Why this matters for the LLM question

Every row above involves reading code, forming a plan, writing helper code, running it, and interpreting crashes. These are tasks LLM agents are being applied to. *Inference:* an agent is most plausibly useful at (a) the search-heavy parts like heap layout experimentation, (b) understanding unfamiliar allocator code, and (c) iterating on crash feedback. It is least obviously useful at guessing a truly random secret, because no amount of reasoning improves a blind 1-in-16 guess. Distinguishing these cases is the heart of the project.

4. Classic mitigations and why they were not enough

MTE and PAC are the third generation of memory defenses. Knowing the earlier generations explains why the industry moved to hardware, and gives you comparison points for Wednesday.

Generation 1: make the easy attacks fail

Mitigation	What it does	How attackers got around it
Stack canary	Places a secret value before the return address and checks it before returning	Leak the canary first, or corrupt data that does not cross it
NX / DEP (no-execute)	Memory is writable or executable, never both, so injected code will not run	Reuse existing code with ROP instead of injecting new code
ASLR	Randomizes where code and data load, so addresses are unknown	Leak one pointer to recompute the layout
Guard pages	Unmapped pages around large allocations fault on overflow	Only helps at page granularity, not for small objects

None of these detect the bug itself. They make *exploitation* harder, and attackers adapted by chaining an information leak with ROP.

Generation 2: control-flow protections

Control-flow integrity (CFI) restricts where indirect jumps and calls may go. **Shadow stacks** keep a protected copy of return addresses. **Pointer Authentication (section 7)** achieves a similar goal using cryptographic signatures stored in the pointers themselves. *Apple says* it was first to deploy PAC, in the A12 Bionic chip in 2018, and that its success in raising exploitation complexity convinced Apple that deep hardware and software integration would be key.

The weakness shared by this generation: they protect *where code goes*, not *what data holds*. Data-only attacks and many heap corruptions pass straight through.

Generation 3: detect the memory error itself

The idea is to catch the out-of-bounds or stale access at the moment it happens. Three tools you will see named together on the Android NDK pages:

- **AddressSanitizer (ASan):** the compiler adds checks around memory accesses, using "shadow memory" to track which bytes are valid. It is accurate but slow and memory-hungry, so it is a testing tool.
- **HWAddress Sanitizer (HWASan):** a software implementation of the tagging idea. Pointers carry a tag and the compiler inserts tag checks. Lower overhead than ASan but still requires recompiling. *Android says* MTE SYNC mode is a faster alternative that does not require recompiling.
- **MTE:** the same tagging idea implemented in hardware, so every load and store is checked by the CPU at low cost.

Background: GWP-ASan, also listed in the NDK menu, is a sampling tool that protects a small random fraction of allocations to find bugs in production. It is cheap because it only checks a few allocations.

The trade-off triangle

Every memory defense balances three things: **coverage** (how many bugs it catches), **cost** (CPU, memory, battery), and **strength** (whether a determined attacker can get around it). Tools that are strong and complete (ASan) cost too much to ship. Cheap tools (canaries) are weak. MTE's pitch is to move that frontier, giving broad coverage at low cost, with the price being that its strength is probabilistic.

Why attackers kept winning

Apple says known mercenary spyware chains against iOS, Windows and Android share one trait: they exploit memory safety vulnerabilities, which it describes as "interchangeable, powerful, and exist throughout the industry." Interchangeable is the key word. If a defense only blocks one bug's exploitation, attackers swap in another bug. A defense must take away a *technique* (such as "overlap a freed object with an attacker object") to matter. That is the framing for evaluating MTE, PAC and Scudo in this project.

5. Allocators and Scudo

The allocator is the library behind `malloc` and `free`, and it is where MTE tagging actually happens. The hardware only checks tags; the allocator decides what tags to assign. That makes allocator policy as important to security as the hardware feature.

What an allocator does

A program asks for N bytes. The allocator finds a free region, hands back a pointer, and remembers enough to take it back when `free` is called. Fast allocators keep **size classes** (separate pools for 16-byte, 32-byte, 48-byte objects and so on) and **free lists** (recently freed blocks, ready for reuse). Reuse is the property attackers exploit: a freed slot is likely to be the next allocation of the same size.

Allocators also store **metadata** (size, state, links between free blocks). If that metadata sits right next to user data, an overflow can corrupt it. Much classic heap exploitation targets allocator metadata. This is why Marius's ISLAB paper is about protecting memory-management metadata.

Scudo: Android's hardened allocator

Background and LLVM docs say: Scudo is a user-mode allocator based on LLVM's sanitizer allocator, designed to add mitigations against heap vulnerabilities while keeping performance reasonable. It is the default on Fuchsia and, per the docs, on Android since Android 11. The name is Italian for "shield."

Its components, as the LLVM docs describe them:

Component	Role
Primary allocator	Fast path for smaller sizes. Carves reserved regions into identical-size blocks, one region per size class.
Secondary allocator	Slower path for large sizes. Uses OS memory mapping, with **guard pages** around each allocation.
Thread-specific data registry	Controls per-thread caches, either exclusive per thread or a shared fixed pool.

Component	Role
Quarantine	Delays reuse of freed blocks to blunt use-after-free. Costly, and **disabled by default** in the documented defaults (`quarantine_size_kb` is 0).

The chunk header and its checksum

Every chunk Scudo hands out is preceded by an 8-byte header. It stores the size class, the chunk's state (available, allocated or quarantined), the allocation type (`malloc`, `new`, `new[]`, `memalign`), size information, an offset back to the underlying block, and a **16-bit checksum**.

The checksum is a CRC32 computed over a global secret, the chunk's own pointer, and the header bytes. The docs say plainly that it is *not intended to be cryptographically strong*, and that corruption is only detected when the header is next accessed. So an overflow that damages a header but never touches it again goes unnoticed.

Do not confuse this checksum with Pointer Authentication. It is a software integrity check in the allocator, not an Arm PAC. Section 7 and the meeting questions return to this, because the sources I read do not say how (or whether) Scudo uses PA.

Randomness and other hardening

Scudo randomizes how blocks are chosen in the Primary allocator, and can randomize how caches are assigned to threads. The docs note that it *trusts the OS and ASLR* for unpredictable page placement, and that security "will be greatly reduced" if that assumption fails. Optional features include zero-filling or pattern-filling memory on allocation, and checks for mismatched allocation and deallocation types (for example `malloc` paired with `delete`).

When Scudo detects trouble it prints an error beginning "Scudo ERROR" and may terminate the process. Messages you will see include *corrupted chunk header* (checksum failed), *invalid chunk state* (typical for a double free), and *misaligned pointer* (alignment is 16 bytes on 64-bit).

How Scudo and MTE connect

Three facts from the Scudo and Android pages:

- `M_MEMTAG_TUNING` is a `mallopt` option that tunes how Scudo picks tags, so that a certain class of memory error is more likely to be caught (details in section 6).
- In SYNC mode, Scudo records the stack trace of allocations and deallocations so fault reports can explain what happened. The LLVM docs describe an `allocation_ring_buffer_size` option (default 32768) controlling how many past allocations are kept for this.
- The sample crash report on the Android NDK page shows frames named like `scudo::Allocator::deallocate` and `quarantineOrDeallocateChunk`. That function name does not mean the quarantine is enabled; the docs say it is off by default.

What to take away

Inference: a Scudo assessment has at least three layers worth separating: (1) allocator logic and its checks, such as header checksum and state checks, (2) MTE tag assignment policy, and (3) the places where tag checking is bypassed or suspended. Marius's plan to look at "how Scudo integrates MTE and PA" probably spans all three. *Ask:* which Scudo revision and which Android build.

6. Memory Tagging Extension (MTE) in depth

MTE is a lock-and-key system for memory: every chunk of memory gets a small "color" (the lock), every pointer to it carries the same color (the key), and the CPU faults when they do not match. This is the single most important section for the project, so read it twice.

The mechanism

- Memory is divided into **16-byte granules**. Each granule has a hidden **4-bit allocation tag** (16 possible values), stored by the hardware outside normal addressable memory.
- A pointer carries a **4-bit logical tag** in its otherwise unused top bits (bits 59 to 56, per the Codex brief and the Linux documentation it cites). The mechanism that lets the CPU ignore those top bits for addressing is called Top Byte Ignore.
- On every load and store, the CPU compares the pointer's tag with the granule's tag. A match proceeds. A mismatch is a **tag check fault**, and what happens next depends on the mode.

Android says MTE is a hardware extension that catches use-after-free and buffer-overflow bugs. *Apple says* each allocation "is tagged with a secret" and the hardware grants access only when the request carries the right one. Both descriptions match the model above.

How the allocator uses it

The NDK guide's section for people writing their own allocator shows what the allocator must do, and therefore what the hardware gives it:

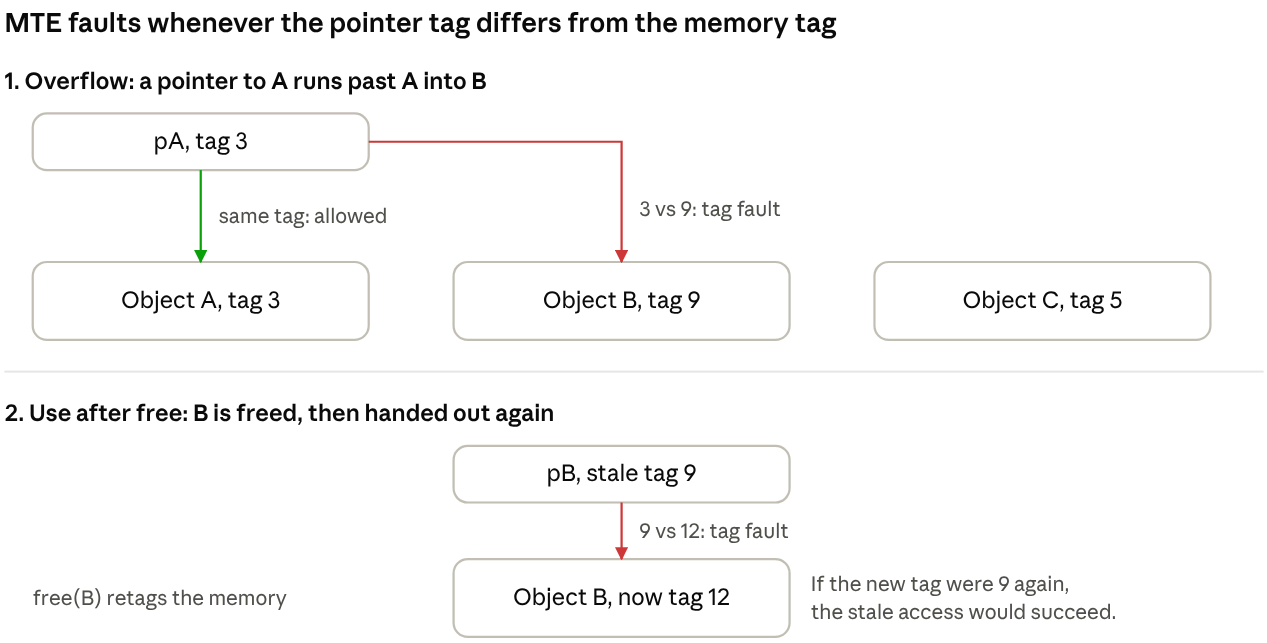
1. Map pages with `PROT_MTE` so the OS enables tagging for them.
2. Align every tagged allocation to 16 bytes, because tags apply to 16-byte granules.
3. Pick a random tag with the `IRG` instruction and place it in the returned pointer.
4. Write that tag onto the memory with `STG` (one granule), `ST2G` (two), or `DC GVA` (a whole cache line). The `STZG`, `STZ2G` and `DC GZVA` variants also zero the memory.
5. On `free`, retag the memory so old pointers no longer match.

A worked example

Three adjacent 32-byte allocations, A, B and C, are given tags 3, 9 and 5. The pointers `pA`, `pB`, `pC` carry those tags.

- `pA[0]` reads a granule tagged 3 through a pointer tagged 3. Match. Allowed.
- `pA[32]` runs off the end of A into B's granule, tagged 9. The pointer says 3, the memory says 9. **Fault.** This is how MTE catches a linear overflow.
- The program frees B. The allocator retags B's memory, say to 12. A stale `pB` still says 9. `pB[0]` now compares 9 against 12. **Fault.** This is how MTE catches use-after-free.
- Now suppose the random re-tag happened to pick 9 again. Then `pB[0]` matches and the stale access succeeds. This 1-in-16 coincidence is the "probabilistic" part.

Apple's article illustrates the same two cases with three adjacent allocations carrying different tags for overflow, and a freed-then-reallocated block carrying a changed tag for use-after-free.



MTE tag check · overflow and use-after-free

A tag of 3 against 9 stops the overflow, and a freed block retagged to 12 stops the stale pointer. If a random re-tag happened to equal the old tag (1 chance in 16), the stale access would go through.

Operating modes

Mode	What happens on a tag mismatch	Typical use
SYNC	The CPU stops at the exact offending load or store. The process gets SIGSEGV with code `SEGV_MTESERR` and full access details.	Testing, and production for high-risk processes. Acts as a real security mitigation.
ASYNC	Execution continues until the next kernel entry (a syscall or timer interrupt), then the process dies with `SEGV_MTEAERR`. The faulting address is not recorded.	Low-overhead production use on well-tested code.
ASYMM (Armv8.7-A)	Synchronous checking for reads, asynchronous for writes. Performance similar to ASYNC.	Android's platform docs prefer it over ASYNC when available.

The ASYNC gap is a security issue, not just a debugging inconvenience. *Apple says* asynchronous reporting leaves a race window for attackers, which is why it insisted on strictly synchronous checking. *Inference:* in ASYNC mode, an attacker may be able to do damage after the bad access and before the process is killed at the next kernel entry.

What is covered, and what is not

Memory region	Tagged by default on Android?	Notes
Native heap (via Scudo)	Yes, when MTE is enabled for the process	Process-wide property.
Stack	No	Needs compile flags (`-fsanitize=memtag` , `-march=armv8-a+memtag`). The NDK page says stack tagging is available from Android 14 QPR3 and builds with instrumentation are for debugging only.

Memory region	Tagged by default on Android?	Notes
Globals / untagged memory	No	Original MTE does not check them. Apple says this is a key weakness that Enhanced MTE addresses.
Kernel	Via MTE-accelerated KASAN	Configured at boot (`kasan.mode=sync` by default).
Inside a single 16-byte granule	Not distinguishable	Same tag, so overflows within a granule are invisible.
Fields inside one allocation	Not distinguishable	Intra-allocation overflows share one tag. Apple's article acknowledges they survive MIE but says they are rare.

How Android enables it

The NDK and AOSP pages describe several switches: `android:memtagMode` in the app manifest (values `off`, `default`, `sync`, `async`), compatibility flags `NATIVE_MEMTAG_SYNC` and `NATIVE_MEMTAG_ASYNC`, build settings in `Android.bp`, a system property `arm64.memtag.process.<name>`, and an environment variable `MEMTAG_OPTIONS`. **MTE is off by default for apps.** *Android says* it is a process-wide property, so enabling it covers every native heap allocation in the process.

To check a device, run `adb shell grep mte /proc/cpuinfo`. The NDK page lists devices from Pixel 8 through Pixel 10a as known to support MTE, and notes some devices offer an experimental developer option to reboot with it enabled.

On the platform side, *Android says* that from Android 12 a set of security-critical system components use MTE in ASYNC mode: networking daemons (except `netd`), Bluetooth, SecureElement and NFC HALs, `statsd`, `system_server`, and `zygote64`.

The tag-assignment policy: the part to study closely

The AOSP page documents a `mallopt(M_MEMTAG_TUNING, ...)` setting with two strategies:

- **`M_MEMTAG_TUNING_BUFFER_OVERFLOW` (default).** Adjacent allocations get *different* tags, so a linear overflow into a neighbor is detected **deterministically**. The cost: only about half of the tag values are available at each location, so use-after-free detection is slightly worse.
- **`M_MEMTAG_TUNING_UAF`.** Tags are independently randomized, giving a uniform **~93%** chance of detecting both overflow and use-after-free.

The 93% figure is just 15 divided by 16, rounded down, so it applies to a single blind access under that policy:

```
P(\text{detect}) = 1 - \tfrac{1}{16} = \tfrac{15}{16} = 93.75\%
```

The lesson: **there is no single "MTE success probability."** Under the default policy, adjacent overflow is deterministic, so guessing is useless there, and the attacker must find a different route. Under the UAF policy, a stale-pointer attack has about a 1-in-16 chance per attempt against a fixed random tag.

Probability, carefully

Under an illustrative model (a hidden tag uniform over 16 values, independent fresh guesses, a match is sufficient), the chance of at least one match in *n* tries is:

```
P(\geq 1\text{ match in }n) = 1 - \left(\tfrac{15}{16}\right)^n
```

That gives about 64% at n = 16, and the expected number of tries to the first match is 16. But the model's assumptions are exactly what real attacks fight over. A failed try usually **crashes the process**, which typically re-randomizes tags. Repeating the same wrong guess against the same fixed tag is not a fresh independent trial. So the real questions are: does the target restart, are tags re-randomized, and can the attacker learn the tag without crashing?

Known weak spots (preview of section 9)

- **Tag leakage** through speculative execution (TikTag, StickyTags).
- **ASYNC windows** for races.
- **Untagged and intra-allocation memory**, where there is no tag to mismatch.
- **Check suppression.** The AOSP page notes that setting `PSTATE.TCO` temporarily suppresses tag checking, for example when copying memory with unknown tags or in a hot loop. This is a legitimate feature, but it is an audit target: can an attacker reach code that runs with checks off?

- **Tag 0.** The Codex brief reports that a reviewed Scudo helper always excludes tag zero from random selection. I did not independently verify this. Check it in the source.

Reading a crash report

The NDK page includes a sample tombstone. The lines to learn to read:

- `signal 11 (SIGSEGV), code 9 (SEGV_MTESERR)` means a synchronous tag mismatch.
- `Cause: [MTE]: Use After Free, 0 bytes into a 32-byte allocation` is Scudo's explanation, available because SYNC mode records stack traces.
- The `deallocated by` and `allocated by` backtraces show where the object was freed and where it was created. These make the bug easy to find, and they are also exactly the type of artifact an LLM agent can consume as feedback.

7. Pointer Authentication (PA, PAC, PAuth)

Pointer Authentication puts a short cryptographic signature inside a pointer, so that a pointer forged or modified by an attacker fails verification before it is used. Where MTE asks "is this memory access allowed?", PA asks "is this pointer the one the program itself created for this purpose?" I read the Linux kernel documentation and Clang's Pointer Authentication manual for this section.

Vocabulary

The terms below come from Clang's manual.

Term	Meaning
Raw pointer	A normal address with no signature
Sign	Compute a signature from the pointer, a key and a discriminator, and store it in the pointer's unused high bits
Authenticate (auth)	Recompute the signature and compare. On success, strip it and return the raw pointer
Strip	Remove the signature without checking it (used by debuggers and crash handlers)

Term	Meaning
Key	A secret held in CPU system registers. The attacker cannot read it from memory
Discriminator	Extra context mixed into the signature (Clang compares it to a *salt*, and the key to a *pepper*)
PAC	The signature itself. "PAuth" is Arm's name for the architecture extension

Conceptually: `signed = sign(address, key, discriminator)`, and `auth(signed, key, discriminator)` returns the address only if the signature matches. This is an integrity check, **not encryption**. The pointer's address bits are still visible.

Why the discriminator matters

Without a discriminator, an attacker could copy any validly signed pointer to any other place that expects a pointer signed with the same key. Mixing in context prevents that. Common choices:

- **Storage address (address diversity):** a pointer stored at address X only validates when read from X, so copying it elsewhere fails.
- **A constant tied to purpose or type:** for example Clang signs C++ virtual functions using a hash of the method's name and signature.
- **The stack pointer:** *Clang says* on Apple's arm64e ABI the return address is signed with the IB key using the stack pointer at function entry as the discriminator.

How big is the signature?

The signature has to fit in the pointer's unused bits, so its width depends on how many address bits the system uses. *Linux docs say* the PAC width is 55 minus the configured virtual address size, and that with a 48-bit address space the PAC is 7 bits wide. *The IUBIK paper* reports a concrete case: on a Pixel 8 running a Linux 5.15 kernel, virtual addresses use bits 0 to 38, MTE tags use bits 56 to 59, and the PAC gets 16 bits (bits 39 to 54). That is a kernel configuration, so userspace may differ. This is where the "7 to 16 bits" range comes from; IUBIK also notes that PAC length can range from 3 to 31 bits depending on hardware configuration. If the signature were uniformly unpredictable and b bits wide, a blind guess succeeds with probability 2 to the power of minus b: 1 in 128 at 7 bits, 1 in 65,536 at 16 bits.

Clang's manual is blunt about this being a tradeoff: storing the signature in high bits keeps pointers the same size and allows interoperability, but "substantially fewer bits" weakens the scheme and makes guessing easier. It also notes that a hash-based scheme with an authentication oracle can make brute force computationally feasible.

Keys

Linux docs say the extension provides five keys: two for instruction addresses (APIAKey, APIBKey), two for data addresses (APDAKey, APDBKey), and one generic key (APGAKey). The kernel assigns random keys to each process at `exec`. The keys are shared across threads in the process and preserved across `fork()`. Userspace can request new keys with the `PR_PAC_RESET_KEYS` option of `prctl`.

The fact that child processes keep their parent's keys after `fork` is a classic reason brute-force attacks on forking servers are interesting: each crash and respawn does not reset the secret. Compare MTE, where tags are chosen per allocation at runtime.

What happens on failure

Baseline Armv8.3 `aut` instructions do not trap by themselves. They corrupt the pointer so that a later use probably faults. With the optional **FPAC** extension, failure traps immediately. *Clang says* this difference matters for security: if the failure only shows up later, and the kernel lets the process recover from the resulting fault, the attacker may be able to build an **authentication oracle** that tells them whether a guess was valid without ending the process.

What it protects, and what it does not

Clang says pointer authentication gives "strong probabilistic protection" against taking control of execution. In Clang's design:

- **Signed by default:** return addresses, C function pointers, C++ virtual functions and v-table pointers, indirect calls through the GOT.
- **Not signed by default:** ordinary C data pointers. Clang says specific sensitive data pointers can opt in with the `__ptrauth` qualifier.
- **Not covered at all:** non-pointer data, such as an `is_admin` flag or a length field. This is why PAC does not stop data-only attacks.

The C function pointer defaults deserve attention. *Clang says* on arm64e they are signed with the IA key, no address diversity, and discriminator 0. That means any validly signed function pointer can be swapped for another, which Clang calls a weakness it addresses optionally with type-based discrimination.

Attack classes from the Clang manual

This list is the best checklist you can take into a PA-focused discussion.

Attack	Idea
Substitution	Overwrite a signed pointer with a different validly signed pointer that uses the same schema
Access path	Don't touch the signed pointer. Replace the pointer you reach it through (for example, an object's v-table pointer)
Signing oracle	Find code that signs an attacker-chosen value, then reuse that signature
Authentication oracle	Find code that reveals whether a signature is valid without crashing
Failure to trap	Auth failure that can be intercepted or recovered from creates an oracle
Attackable code sequences	A raw pointer sitting in memory or a spilled register between steps of a sign operation can be overwritten
Signature forging	Reproduce the signing algorithm, or learn the key
Remapping	Change page protections so injected code can act as an oracle

The threat model stated in Clang's manual is also worth knowing. The attacker can **read and write memory**, possibly the whole address space, but cannot add new executable code. A bug that simply crashes the program is accepted as the best-case outcome. Notice how that differs from MTE's model, where the attacker's access is what gets checked.

MTE and PA side by side

	MTE	PA
Protects	Whether a memory access is permitted	Whether a pointer value is genuine
Secret lives in	Tag memory and the allocator	CPU key registers
Check happens	On every tagged load and store	When an authenticated pointer is used (return, call, load)
Strength	4 bits, probabilistic	About 7 to 16 bits, probabilistic
Catches bugs	Yes, the out-of-bounds or stale access itself	No, only the later misuse of a corrupted pointer
Typical stage in an attack	Corrupt	Hijack

Where PA could enter the Scudo story

The pages Marius linked establish MTE support in Scudo, not PA. I found no mention of Pointer Authentication in the LLVM Scudo documentation. *Background:* Android's platform binaries are built with return-address protection on PAC-capable devices (verify the exact build flags and which release). So the PA in "MTE and PA" might be (a) compiler-generated return-address signing in the allocator and its callers, (b) PA used by a separate layer, or (c) something Marius has in mind that is not in the two pages. Ask which (see section 12).

PACMAN, briefly

Apple says evaluating a PAC instruction speculatively exposed timing differences in its original implementation, which would let an attacker isolate the valid signature. This is the PACMAN attack (section 9). It matters here because it is a validity oracle that does not crash the program, which removes the main cost of guessing.

8. Apple's Memory Integrity Enforcement (MIE)

MIE is Apple's answer to a specific question: what does it take to turn MTE from a debugging aid into a security feature that holds up against a determined attacker? Read Apple's post as a catalog of every weakness in baseline MTE, paired with the fix Apple chose. This section summarizes the post, which was published September 9, 2025.

Timeline from the post

When	What	Why it matters
September 2025	MIE ships in all iPhone 17 models and iPhone Air (A19 and A19 Pro chips)	First always-on, synchronous, hardware-tagged memory safety across the kernel and many userland processes
2022	Arm releases the Enhanced MTE (EMTE) specification, developed with Apple input	Fixes shortcomings Apple found in the original design
2019	Arm publishes the original MTE specification	The starting point
iOS 17	xzone malloc, the userland typed allocator	Type-aware heap placement for apps
iOS 15	kalloc_type, the kernel typed allocator	Same idea in the kernel
2018	Pointer Authentication Codes arrive on the A12 Bionic	Control-flow protection; showed Apple that hardware-software integration works

The three layers

Apple says MIE is built from three parts that are designed together.

Layer 1: secure typed allocators (kalloc_type, xzone malloc, WebKit's libpas). These allocators use the *type* of each allocation to decide where it lives. Apple's reasoning: in both use-after-free and out-of-bounds exploits, the attacker wants two different

interpretations of the same bytes, which they get by controlling where objects of a chosen type land. If objects of different types never share memory, many such techniques fail before MTE is even involved.

The catch is granularity. Apple says allocators can only apply protections at the **page** level (16 KB on iOS), which works across type buckets and for large allocations, but is too coarse to separate small objects *within* one bucket. That gap is what memory tagging closes.

Layer 2: Enhanced MTE in synchronous mode. Within a bucket, tags catch overflows and use-after-free exactly as described in section 6. Apple makes three design choices worth noting:

- **Strictly synchronous.** Apple says memory safety protection must be on by default and working continuously, and it rejects asynchronous reporting because of the race window.
- **Distinct tags for neighbors.** The allocator uses different tags for adjacent allocations so a linear overflow always faults, and retags memory on reuse so stale pointers fail.
- **Non-tagged memory is no longer a free pass.** Apple says original MTE does not check accesses to untagged memory such as global variables, so attackers can corrupt core application state without facing tag checks. In EMTE, reaching untagged memory *from a tagged region* requires knowing that region's tag.

Layer 3: Tag Confidentiality Enforcement (TCE). MTE and typed allocators both depend on secrets staying secret: the tags, and the kernel's allocator data structures. Apple lists the measures:

- The Secure Page Table Monitor protects the allocator's backing store and tag storage, with guarantees even after a kernel compromise.
- When the kernel touches memory on behalf of an app, it obeys the same tag-checking rules as userspace.
- Three speculative-execution threats are addressed individually (below).

The three speculative threats Apple addresses

Threat	What the attacker does	Apple's response
Tag checks leak through timing	Use speculative execution to learn whether a tag guess matched, without crashing	Designed the silicon so tag values cannot influence speculative execution at all. Apple says published work (StickyTags, TikTag) shows this attack works against MTE on Google Pixel devices
Tags are predictable	Predict the next random tag the allocator will pick	Frequently re-seed the pseudo-random generator that chooses tags
Spectre V1	Abuse conditional branches to leak data, including tags	A low-cost mitigation that forces attackers to deal with type segregation. Apple says attackers would typically need 25 or more V1 sequences to exceed 95% exploitability, unless one relates to the bug itself

Cost and hardware

Apple says it spent unusually large amounts of silicon (CPU area, CPU speed, memory for tag storage) on MIE. It also designed the system to use the typed allocators first and spend tag-checking only on the small allocations within buckets that software cannot defend alone, then sized the hardware to that modeled demand. *Inference:* this is the "coverage, cost, strength" triangle from section 4 again. Apple bought strength by spending cost, then reduced cost by narrowing coverage to where tags are needed most.

What Apple's evaluation claims

Apple's offensive research team worked on the system from 2020 to 2025. They rebuilt previously seen exploit chains against MIE, and also examined recent vulnerabilities expected to have the best chance of surviving, enumerating all exploitation opportunities.

Apple says it could not rebuild any of those chains around MIE, that the surviving effects were unreliable, and that MIE tends to stop attackers early, when their capabilities are still limited. It illustrates this with six real chains and shows which steps MIE blocks.

Two honest caveats from the post itself: (1) **intra-allocation overflows survive**, though Apple calls them extremely rare and unlikely to support a full exploit, and (2) "there's no such thing as perfect security."

Deployment scope

Apple says MIE covers the kernel and over 70 userland processes on iPhone 17 and iPhone Air. It also made EMTE available to third-party developers through the Enhanced Security settings in Xcode, because apps like messengers and social networks are likely entry points.

How to read this critically

- **It is a vendor's evaluation of its own product.** Apple's red team tested selected chains. That is evidence, not proof that all attacks fail.
- **The Android comparison is Apple's framing.** Apple says Android's MTE is limited by missing deep OS integration, and that Google's MTE is opt-in. Check any such claim against specific devices and Android versions before repeating it.
- **Baseline MTE is not MIE.** A result showing a bypass of Android's baseline setup says little about MIE, and vice versa. The Codex brief makes the same point.
- **"Defeating a chain" is not the same as "defeating MIE."** Apple frames success as raising the cost of the chain. Keep that economic framing in mind for the LLM question: an agent that makes some attack step cheaper is changing the economics, even if it does not break the hardware.

Questions to hold while you read the post

1. Which Apple measure addresses which weakness in my section 6 list?
2. Which of Apple's measures depend on Apple controlling the whole stack, and which could Android adopt?
3. If an LLM could do a certain step cheaply, which of Apple's layers would still stop the chain?

9. Known attacks: PACMAN, TikTag, StickyTags

The published attacks on PAC and MTE share one idea: make a wrong guess cost nothing. If an attacker can test a guess without crashing the process, a 1-in-16 or 1-in-65,536 chance stops mattering, because they can simply test every possibility. I read the PACMAN project page, the TikTag abstract, and the StickyTags project page for this section. Treat the table as a reading list, not a substitute for the papers.

The three papers at a glance

Paper	Target	Core technique	Headline result	Authors' caveat
PACMAN (MIT CSAIL, ISCA 2022)	Pointer Authentication on Apple M1	Speculative execution used as a PAC validity oracle, read out through a side channel on the TLB	Forge kernel PACs from userspace on M1. Arm later reported many of its own cores are also affected	Needs an existing memory bug. The authors say it is an exploitation technique and cannot compromise a system alone
TikTag (arXiv, June 2024)	MTE on Google Pixel hardware	Gadgets that leak the tag of an arbitrary address through speculation	Tag leak success above 95% in under 4 seconds. Demonstrated against Google Chrome and the Linux kernel	The abstract also proposes defenses

Paper	Target	Core technique	Headline result	Authors' caveat
StickyTags (VUSec)	MTE tagging policy and speculative probing	(1) A deterministic round-robin tag layout that never retags. (2) A disclosure that a contention side channel reveals tag mismatches	Deterministic layout cuts overhead to about 4% (heap and stack) versus about 20% for Scudo plus MemTagSanitizer on a Pixel 8 Pro	Disclosed to Arm, which published an advisory about speculative oracles on memory tagging

How PACMAN works, step by step

The PACMAN authors describe a clean recipe that generalizes beyond PAC:

1. Start with an ordinary memory read/write bug. It lets the attacker supply a *guess* for the PAC on a pointer.
2. Find a **gadget** in the victim: code that authenticates a pointer and then uses it, but only on a conditional path the attacker can steer.
3. Run the guess **speculatively**. The CPU executes the path ahead of time, and if the guess was wrong, it throws the work away without raising a visible crash.
4. Read the result through a **side channel**. PACMAN watches whether the speculative load evicted entries from the TLB (a technique called prime+probe). Eviction suggests the PAC was right.
5. Repeat for every possible PAC value. The PAC space is small, so brute force is feasible.

The authors state the attack "leaves no logs" because it runs entirely in the speculative regime. The authors also note Apple knew since 2021. *Apple says* in the MIE post that its original PAC implementation showed the speculative timing differences this attack relies on.

How TikTag and StickyTags apply the same idea to MTE

The pattern is identical, with a tag in place of a PAC. *TikTag abstract:* gadgets can leak MTE tags from arbitrary addresses through speculative execution, letting attackers bypass the probabilistic defense and raise their success rate close to 100%. *StickyTags page:* a contention-based side channel reveals whether a tag check produced a mismatch, and

building a probe gadget is relatively simple. It needs the target bug on a speculative path plus any observable signal from an independent memory operation in the speculation window.

The StickyTags page adds a pointed observation: if attackers can learn the tags at runtime, the hardware's random source gives no added benefit. That sentence is the best one-line summary of why tag confidentiality matters.

What StickyTags teaches about tag policy

StickyTags is also a design paper. It points out that Scudo-style random tagging requires frequent retagging, which is expensive (it reports 5.8% overhead on the heap, and about 15% for stack tagging with MemTagSanitizer). Its alternative assigns tags round-robin to size-class slots and leaves them in place across object lifetimes. That gives deterministic protection against overflows bounded by tag count times slot size.

Inference: this shows that "random" versus "deterministic" tag choice is a real design axis, which feeds directly into section 6's discussion of `M_MEMTAG_TUNING`. A deterministic layout is great against linear overflows and weak against stale pointers to reused slots (the tag never changes when a slot is reused, so a dangling pointer still matches). *This is my reading of the design, not something the page states in those words.* Confirm it in the paper.

Taxonomy: ways defenses like these fail

This list is my synthesis from the sources above. Use it as a checklist when you look at any hardening technique.

Failure mode	Example	Is it "guessing"?
Lucky or brute-force guess	1-in-16 tag collision	Yes
Validity oracle that avoids crashes	PACMAN, TikTag, StickyTags probing	Removes the cost of guessing
Defense not applied to the target	Untagged globals, unsigned data pointers, stack without instrumentation	No
Policy gap in the allocator	Tag reuse, excluded tags, deterministic patterns	Sometimes

Failure mode	Example	Is it "guessing"?
Check disabled or delayed	ASYNC mode window, `PSTATE.TCO` suppression	No
Granularity gap	Overflow inside one 16-byte granule or one allocation	No
Substitution of a valid value	Swapping one signed pointer for another with the same schema	No
Signing or authentication oracle	Code that signs attacker-chosen values	No

How this feeds the LLM question

The first row is the only one where "a smarter attacker" cannot help, because information-free guessing is just luck. Every other row requires *finding* something: a gadget, a coverage gap, a policy gap, a reachable suppression path. Finding things in large, unfamiliar code is precisely what the LLM agent literature is trying to measure. That is the cleanest framing of Marius's hypothesis I can offer, and it is mine, so test it with him.

A sensible baseline for any evaluation is whether an agent can rediscover and reproduce what these three papers did, under a stated threat model and budget. Success there says the agent can execute known strategies. It does not yet say it can discover new ones.

10. The research question: do LLMs change the math?

The strongest version of Marius's question is not "can an LLM beat a 1-in-16 guess?" (it cannot, by reasoning alone) but "does an agent change the cost, reliability, or range of exploitation strategies against a precisely specified defense?" Everything in this section is a proposal for discussion. Marius's message gave you the topic and the starting target; the experimental design below is my synthesis and the Codex brief's, not an agreed plan.

Why scripts already beat guessing, and what agents might add

A plain script can try every one of 16 tags or 65,536 signatures. A blind brute-force attack needs no language model. So the interesting question is where the **search space is semantic rather than numeric**: reading unfamiliar allocator code, spotting that some path leaves memory untagged, writing and debugging a probe gadget, noticing a crash trace implies a tag collision.

A useful way to organize the hypothesis is as a chain of stages, each with its own success probability. An agent can matter only where it moves one of the factors:

```
P(\text{exploit}) = P(\text{trigger}) \times
P(\text{primitive}\mid\text{trigger}) \times
P(\text{bypass}\mid\text{primitive}) \times P(\text{effect}\mid\text{bypass})
```

Stage	Could an agent plausibly change it?	Why
Find or trigger the bug	Yes, possibly	Code reading, fuzz-harness writing (Marius's Revelio targets this for C)
Build a primitive	Yes, possibly	Heap layout search is trial-and-error with feedback
Bypass the defense	Depends on the route	Blind guess: no. Finding a leak or coverage gap: maybe
Turn it into an effect	Yes, possibly	Writing payload code, debugging

Inference. These are hypotheses. None is established, and part of the project's value is testing them honestly, including finding that the answer is "no."

Candidate research questions

- **RQ1.** How much does each enabled defense (MTE SYNC, MTE ASYNC, PAC) reduce an agent's success compared with the same tasks undefended?
- **RQ2.** Under equal information and budget, how do agents compare with a scripted random search, a conventional fuzzer, and (if feasible) a human expert?
- **RQ3.** When an agent succeeds, what is the mechanism: random collision, a leak, a pointer substitution, an allocator-policy gap, an unprotected target, or a delayed fault?
- **RQ4.** How much does access matter: source code, debug info, runtime feedback, published attack descriptions?
- **RQ5.** Do successful strategies transfer across fresh random seeds, builds, and related programs?

Write the threat model before launching an agent

Dimension	Decisions to record
Starting access	Remote input, local app code execution, or another foothold
Information	Source, binary, symbols, crash traces, allocator internals
Bug status	Bug and trigger given, location given, or independent discovery required
Allowed capabilities	Debugger, memory reads, timing measurements, restarts
Target	User-space program, allocator internals, or kernel
Budget	Time, tokens, tool calls, executions, crashes, dollars
Success	The exact security boundary crossed, and observable evidence
Deployment	Hardware, OS build, allocator revision, compiler, defense settings

A debugger that reads tags is fine during development. If the final attack depends on it, the result belongs in a different category. An agent that disables MTE or edits the binary has not bypassed the original configuration.

Define success carefully

Use the five-stage vocabulary from section 2 and report **at least three separate rates**: bug-trigger rate, verified protection-bypass rate, and end-to-end security-effect rate. Always give denominators, and distinguish *agent runs* from *individual executions* of the target. A crash is not a bypass. An application-level permission change, on the other hand, can be a real security effect even without hijacking execution.

Observation	Defensible conclusion
Agent triggers a tag fault	It reached a bug and the defense activated

Observation	Defensible conclusion
One blind attempt matches the tag	A collision occurred under that trial's conditions
Agent reproduces TikTag on the specified setup	It reproduced a known strategy there
Agent corrupts an unprotected field	A coverage limitation enabled that specific effect
Agent succeeds more often at fixed budget	Evidence of an empirical advantage on the evaluated tasks
Agent fails within budget	That agent and setup failed within that budget. It says nothing about all agents

Measurement hygiene

- **An oracle outside the agent's control** decides success, so the agent cannot grade itself.
- **Replay claimed successes** from a clean state and with fresh random seeds.
- **Keep the failures.** Report the full distribution, not the best run.
- **Uncertainty.** With small samples, intervals are wide. For example, 3 successes in 20 runs gives a 95% Wilson interval of roughly 5% to 36%, which is too wide to claim a rate.
- **Cost accounting.** Report time, crashes, tool calls and dollars alongside success.
- **Reproducibility.** Preserve prompts, model versions, tool transcripts, compiler flags, hardware identifiers and seeds.
- **Contamination.** Published attacks may appear in the model's training data. Separate *reproducing known attacks* from *held-out tasks* that test discovery and transfer.

Baselines worth proposing

1. An LLM with no execution tools (answers from knowledge only).
2. The full agent with tools.
3. A scripted random-search baseline.
4. A conventional testing baseline such as fuzzing.
5. An expert baseline, only if it is feasible and honestly described.

Equalize information and access where you can. If budgets cannot be equalized, report both time and money.

Start narrow

The Codex brief recommends one controlled Android user-space target on real MTE-capable hardware, with Scudo and SYNC mode verified, beginning with small, deliberately vulnerable C programs whose expected behavior you can check. Validate the harness first, then extend. Add PA only after you confirm what it covers in that build. Emulation is fine for harness development, but it says nothing about physical speculative side channels.

Ask Marius: is this the right order, and where does his own Revelio or Crustify harness plug in?

11. Marius Momeu and his papers

Marius is a security researcher who built hardware-assisted defenses (including one with MTE) and is now applying LLM agents to systems security. That combination is exactly the project: he knows the defenses from the builder's side, and he now wants to stress-test them from the agent's side. Everything here is from his personal website, which I read today, and from the IUBIK paper, which I read in full.

Who he is

- Postdoc in the Sky Computing Lab at UC Berkeley, in Koushik Sen's group. His site's news feed says he started April 1, 2026.
- PhD *summa cum laude* from TU Munich (Chair of IT Security, advisor Claudia Eckert), defended March 27, 2026. Also affiliated with Brown University, working closely with Vasileios Kemerlis's group.
- Stated goal: make systems software reliable and secure by preventing and detecting flaws before attackers exploit them.

Current projects (from his site)

Project	What it is
Crustify	An LLM agent harness for Rust tasks: migrating C/C++ codebases to Rust, generating safe Rust bindings over unsafe APIs, and finding undefined behavior and memory-safety bugs in Rust. Part of DARPA's TRACTOR program (translating legacy C to Rust)
Revelio	An agent harness for finding and reproducing security-critical memory-safety vulnerabilities in C code
KISS Sorcar	A general-purpose agent framework for long-horizon tasks (its code repository appears to live under an account named ksenxx; I did not confirm whose)

Inference: Revelio is probably the closest existing tool to what the project needs, since it already handles finding and reproducing memory bugs. Ask whether the new work would build on it.

Earlier work: hardware-assisted hardening

Paper	Venue	One-line summary	Hardware
HyperMirage	NDSS 2026	Direct state manipulation in hybrid virtual-CPU fuzzing of hypervisors	n/a (fuzzing)
IUBIK	IEEE S&P 2025	Isolating user-controlled bytes in OS kernels via MTE	Arm MTE plus PA
SafeSlab	ACM CCS 2024	Mitigating use-after-free in kernel heaps	Intel memory protection keys
ISLAB	ACM AsiaCCS 2024	Making memory-management metadata immutable in kernels	Intel SMAP
xMP	IEEE S&P 2020	Selective memory protection for kernel and user space	Intel EPT (virtualization)

Notice the pattern: each paper repurposes a hardware feature to enforce *isolation*. That is a different philosophy from random tagging, and it matters for how he may frame the new project.

Read IUBIK first

IUBIK is the paper to read before Wednesday, because it is the only one that uses MTE and PA together. Here is what it does, in plain language.

The observation. The authors analyzed kernel exploits from 2020 to 2024 and found most share a trick: use a limited memory bug (use-after-free or out-of-bounds) to overlap a **user-controlled object** with a sensitive kernel object. User-controlled objects, which the paper calls *usercopy primitives*, are objects the attacker can fill with chosen bytes through calls like `copy_from_user`. Instead of trying to list every sensitive object to protect (which keeps failing), IUBIK isolates the attacker-controlled objects.

The mechanism.

1. **Two MTE domains.** Kernel heap pages are tagged with one of exactly two tags: TagU for user-controlled objects, TagK for everything else. Access across the boundary faults. The paper uses synchronous mode. Because it needs only two tags, it avoids the limitation of having just 16 tags, which it says raised concerns in prior work, including StickyTags.

2. **Struct rewriting.** Many kernel structs mix attacker-controlled bytes with sensitive fields such as function pointers. IUBIK splits such a struct into two objects, putting the user bytes in the TagU domain and the sensitive rest in TagK. The paper reports rewriting 29 structs in total.
3. **A PAC-protected back-reference.** When code needs to go from the extracted part back to the original object (the `container_of` idiom), the paper keeps a backward pointer and protects it with PA: signed with `PACDA`, checked with `AUTDA`, using the pointer's storage address as context to stop replay. This is the "PA" use in this paper.
4. **A profiler.** A dynamic tool finds allocation sites where user data lands. It recorded 292 privileged and 212 non-privileged allocation sites across the workloads they ran.

The results. On a Pixel 8 with an Android 14 kernel (Linux 5.15), the prototype showed worst-case slowdowns of 7% to 8% on two micro-benchmarks and under 3% on macro-benchmarks, with 2.17% more memory at boot (about 7 MB). In a survey of 31 public exploits from the previous five years, the paper says 28 (90.3%) would be mitigated. Three would not, for reasons the authors discuss.

The scope and the caveats that matter to you. Pay close attention to the threat model section.

- The attacker can trigger memory errors as often as they like, but **side-channel, microarchitectural and fault-injection attacks are out of scope**. That is exactly where TikTag and PACMAN live.
- The design **requires that MTE and PA cannot be disabled**.
- The authors list weaknesses they have not seen in any exploit: attackers might overlap pointers between the two domains, corrupt them, or use a write primitive on a sensitive object (such as process credentials) obtained without usercopy primitives.
- The conference meta-review noted possible bypasses had not been demonstrated, and that profiler accuracy is not guaranteed.
- The authors suggest the idea could port to user space, with the Heartbleed bug (a heap over-read in OpenSSL) as an example of what isolating attacker-controlled input could prevent.

Inference: IUBIK is a deterministic isolation policy built on MTE, rather than relying on random tags to hide secrets. That makes it an interesting second target: an agent evaluation could ask whether it finds paths the authors list as weaknesses.

Questions to ask about his work

1. Does this project primarily reassess random tagging in Scudo, or also structured isolation policies like IUBIK's?
2. Is the starting point the Android user-space allocator, or the Linux kernel (where IUBIK and his earlier papers live)?
3. How does Revelio relate to the proposed agent harness?
4. Are side channels in scope for this project, given IUBIK treated them as out of scope?

12. Wednesday meeting prep

The goal of a first "touch base" with a postdoc is not to impress him with knowledge. It is to show you did the reading, understand the question well enough to ask good ones, and leave with a small, concrete first task and a check-in date. The Codex brief lists the meeting as Wednesday, October 7 at 11 AM Pacific; I could not verify that, so check your calendar.

Your honest framing

You are new to security, and Marius introduced you through Koushik Sen, so he likely expects that. Say it plainly, then show your work. Something like:

> "I'm new to security, so I spent the last couple of days building a map. I read the Apple MIE post and the Android MTE guide, and started on IUBIK. My understanding is that the question is whether agents change the cost or reliability of exploiting MTE and PA, through finding leaks and coverage gaps rather than guessing tags. Before I go further, I'd like to pin down the exact Scudo version, where PA enters, and what counts as success."

Adjust the words to your voice. The point is that it signals you understand why random guessing alone is not the interesting case.

The five questions to prioritize

If the call is short, get these answered. They decide what you actually do next.

1. **What is the first deliverable?** A literature review, an allocator source audit, a reproduction of known attacks, or an agent evaluation harness?

2. **Which exact Android target?** Device, OS build, Scudo revision, tagging policy (`M_MEMTAG_TUNING` setting), SYNC or ASYNC.
3. **Where does PA enter?** The Android pages he sent cover MTE. Is PA about the allocator's own return addresses, an additional layer, or something in a proposed design?
4. **What does the agent start with?** A known bug, the source, a binary, a debugger, crash reports, or only an input interface?
5. **What counts as success?** A tag mismatch evaded, unauthorized data changed, control flow redirected, or a full chain?

Second-tier questions

- Are side channels (TikTag, StickyTags, PACMAN-style) in scope? IUBIK treated them as out of scope.
- What comparison is intended: humans, fuzzers, scripted guessing, or different agent designs?
- How will we separate reproducing public attacks from discovering new ones, given training-data contamination?
- Is IUBIK background reading or part of the target scope?
- Does this build on Revelio or Crustify, or on a new harness?
- Is there hardware available (a Pixel 8 or later), or are we starting in emulation?
- Who decides what gets disclosed to Google, and when?
- What should I accomplish in the first week, and when is the next check-in?

Things you can offer

You do not have to promise novel attacks. Concrete, useful, honest contributions for someone new:

- **Harness engineering:** a reproducible setup that runs a target, logs everything, and checks success independently.
- **Trace analysis:** parsing tombstones and fault reports into structured records an agent can use.
- **Source auditing:** tracing `malloc`, `free` and `realloc` through Scudo's tagging paths at a pinned revision and recording tag exclusions, reuse behavior, and every place checks are skipped or suspended.

- **Literature table:** the attack-versus-defense-versus-assumption matrix from section 9, filled in properly.
- **Test suite:** small deliberately vulnerable programs plus expected faults, to verify the defenses behave as claimed.

What not to claim

- Do not say "MTE is 1 in 16" without qualification. Say the effective probability depends on the allocator policy and on whether failures crash and re-randomize.
- Do not conflate Scudo's header checksum with PAC.
- Do not say Android's baseline MTE results transfer to Apple's MIE, or vice versa.
- Do not treat a crash or a tag fault as a successful exploit.
- Do not present vendor claims (Apple's, Google's) as independent measurements.

A self-test before you go

You are ready when you can answer these out loud without notes. Ask me for a worked answer to any you cannot.

- What is the difference between a spatial and a temporal memory error? Give an example of each.
- How does MTE catch a use-after-free? Why can it miss one, and with what probability?
- Why is ASYNC mode weaker than SYNC mode from a security view?
- What does `M_MEMTAG_TUNING_BUFFER_OVERFLOW` guarantee that `M_MEMTAG_TUNING_UAF` does not, and vice versa?
- What is a PAC, what is it computed from, and what does it protect?
- Why did PACMAN need a memory bug, and why does an oracle that does not crash matter?
- Name two ways MTE can fail that have nothing to do with guessing.
- What three layers make up Apple's MIE, and which weakness of baseline MTE does each address?
- What did IUBIK use PA for, and what did it use MTE for?
- What is the difference between a bug, a primitive, and a security effect?

First-week plan to negotiate

- Day 1 to 2: one-page threat model and target configuration sheet (use the table in section 10).
- Day 2 to 3: read IUBIK's introduction, threat model and limitations, then skim Arm PA and MTE docs alongside it.
- Day 3 to 5: audit the chosen Scudo revision's allocation, deallocation and tagging paths.
- Day 4 to 6: build a minimal test suite showing valid accesses succeed and representative violations fault as expected.
- Day 6 to 7: write the external success oracle and structured logging, then bring results and uncertainties to Marius before any agent is added.

Notes space for the call

Topic	What Marius said	Follow-up for me
First deliverable		
Target (device, build, Scudo revision)		
Where PA enters		
Agent's starting information		
Success criterion		
Side channels in scope?		
Next check-in date		

13. Glossary

Alphabetical. When a word trips you up in a paper, check here first, then ask me.

Term	Plain-language definition
Address diversity	Mixing a pointer's storage address into its PAC so the signed pointer only validates at that location
Allocation tag	The 4-bit tag stored with a 16-byte granule of memory under MTE

Term	Plain-language definition
Allocator	The library behind `malloc` and `free` that decides where heap objects live
Allocsite	A place in code that allocates memory (IUBIK's term)
ARM / AArch64	The processor architecture used in phones and Apple silicon. AArch64 is its 64-bit mode
ASLR	Address space layout randomization: shuffling where code and data load
ASYMM	MTE mode with synchronous read checks and asynchronous write checks
ASYNC	MTE mode that reports a mismatch later, at the next kernel entry
Authentication oracle	Any code that reveals whether a PAC is valid without crashing
Bypass	Defeating a defense under stated assumptions. Define it before claiming one
Canary (stack)	A secret value placed before a return address and checked before returning
CFI	Control-flow integrity: restricting where jumps and calls may go
Chunk	One allocation as managed by an allocator, including its header
Compartmentalization	Splitting memory into domains that cannot freely access each other
Control-flow hijack	Redirecting where a program executes next
CVE / CWE	A specific public vulnerability / a category of weakness
Dangling pointer	A pointer to memory that has been freed
Data-only attack	Changing security-relevant data without altering control flow

Term	Plain-language definition
Discriminator	Context value mixed into a PAC (acts like a salt)
EMTE	Enhanced MTE, Arm's 2022 specification developed with Apple's input
Exploit	Code or input that turns a bug into attacker benefit
Exploit chain	Several exploits and bugs combined to reach a goal such as full device takeover
FPAC	Optional Arm extension that makes a failed PAC authentication fault immediately
Gadget	A short code sequence in the victim that an attack repurposes (ROP gadget, speculative gadget)
Granule	The 16-byte unit that carries one MTE tag
Guard page	An unmapped page placed next to an allocation to fault on overflow
Heap	Memory requested at run time with `malloc` or `new`
Heap grooming (feng shui)	Arranging allocations and frees to get a desired memory layout
Intra-allocation overflow	Overflow from one field into another inside the same allocation. Tags cannot see it
IRG	Arm instruction that inserts a random tag into a pointer
KASAN	Kernel AddressSanitizer. MTE-accelerated KASAN uses hardware tags in the Linux kernel
Logical tag	The tag carried in a pointer (the "key")
MIE	Memory Integrity Enforcement, Apple's combined defense
Mitigation	A defense that makes exploitation harder without removing the bug
MTE	Memory Tagging Extension: Arm's hardware tagging of memory and pointers

Term	Plain-language definition
Oracle	Anything that answers a question about a hidden value
OOB	Out-of-bounds access (spatial error)
PAC / PAuth / PA	Pointer Authentication Code, and Arm's Pointer Authentication extension
Primitive	A useful capability an attacker gains from a bug, such as an arbitrary write
PROT_MTE	Flag used when mapping memory so the OS enables tagging on those pages
Quarantine	An allocator feature that delays reuse of freed memory
ROP	Return-oriented programming: chaining existing code fragments ending in returns
Sandbox	An environment that restricts what code can do
Scudo	The hardened allocator used by Android and Fuchsia
Shadow memory	Separate memory used to hold metadata, or (in IUBIK) isolated attacker-controlled data
Side channel	Learning secrets from indirect effects such as timing or cache state
Signing oracle	Code that signs an attacker-chosen pointer
Size class	A pool of same-sized allocations inside an allocator
Spatial error	Access outside the intended object's bounds
Speculative execution	A CPU running instructions ahead of time before knowing they are needed
Stack	Memory for local variables and return addresses
STG / ST2G	Arm instructions that write an allocation tag to one or two granules

Term	Plain-language definition
SYNC	MTE mode that faults at the exact offending access
Tag Confidentiality Enforcement	Apple's set of measures that keep MTE tags secret, including from speculative attacks
Tag mismatch	The pointer's tag and the memory's tag differ, causing a tag check fault
TBI	Top Byte Ignore: the CPU ignores the pointer's top byte when addressing, allowing tags to live there
Temporal error	Use of memory outside its lifetime, such as use-after-free
Threat model	The written assumptions about attacker abilities and goals
Tombstone	Android's crash report file
Typed allocator	An allocator that separates allocations by type (Apple's kalloc_type and xzone malloc)
UAF	Use-after-free
Usercopy primitive	A kernel object the attacker can fill with chosen bytes via calls like `copy_from_user`
Zygote	Android process from which apps are forked, so its settings affect apps

14. Reading order, sources, and your notes

Read the two pages Marius sent first, then IUBIK, then the rest as questions arise. Time estimates are rough guesses for a newcomer reading carefully.

Suggested reading order

Order	Item	Rough time	Why
1	This guide, sections 0 to 4	45 min	Vocabulary and mental models

Order	Item	Rough time	Why
2	Android NDK: Arm MTE	20 min	Marius's practical starting point. Study the modes and the sample tombstone
3	Apple: Memory Integrity Enforcement	30 min	Marius's systems-design reading. Map each Apple measure to a weakness of baseline MTE
4	This guide, sections 5 to 8	45 min	Allocators, MTE, PA, MIE in depth
5	IUBIK paper (introduction, threat model, Section 5.1, limitations)	60 min	Marius's own MTE and PA design
6	AOSP: Arm MTE (platform)	20 min	Tag-tuning strategies and enablement controls
7	LLVM: Scudo Hardened Allocator	20 min	Allocator structure, header, options
8	Clang: Pointer Authentication, "Theory of operation"	30 min	Attack classes on PA
9	PACMAN, TikTag and StickyTags (links below)	90 min	Threat models first, then techniques
10	This guide, sections 10 to 12	30 min	Research design and meeting prep

Sources I opened on October 6, 2026

- Apple Security Research: Memory Integrity Enforcement (published September 9, 2025)
- Android NDK guide: Arm Memory Tagging Extension (page last updated July 8, 2026)
- AOSP: Arm Memory Tagging Extension (page last updated September 30, 2026)
- LLVM: Scudo Hardened Allocator
- Linux kernel: Pointer authentication in AArch64 Linux
- Clang: Pointer Authentication
- PACMAN project page
- TikTag on arXiv (abstract)

- StickyTags project page (VUSec)
- Marius Momeu's website
- IUBIK, IEEE S&P 2025 (author-hosted PDF)

In the Codex brief but not opened by me

I have not independently checked these, so treat any claim that rests only on them as unverified: the Linux MTE documentation, MITRE CWE-787 and CWE-416 pages, the TUM publication list, and the AOSP `memtag.h` source snapshot (including its claim that tag zero is always excluded). The first two are standard references. The last one is worth looking at yourself when you start the allocator audit.

What I am least sure about

- **PA inside Scudo.** None of the pages I read say how, or whether, Scudo uses Pointer Authentication. Treat it as an open question for Marius.
- **Exact details of TikTag and PACMAN beyond their abstracts and project pages.** Read the papers' threat-model sections before quoting numbers.
- **Device details.** Which Android devices have MTE enabled by default, and which allocator tuning setting they ship with, changes between releases. Confirm on the specific device.
- **The meeting time.** It comes from the Codex brief only.

Your question log

Use this table as you read. Bring the unresolved rows to me, and the ones I cannot answer to Marius.

Section	What tripped me up	Answer or follow-up